NEXUS

Plateforme de collaboration interentreprises

Équipe
Latrech Arij
Ben Boubaker Sarra
Ben Hamouda Yoser
Dawahi Med Aziz
Mazitti Chaima
Mihoubi Akrem

2026-2027

Sommaire

1.Présentation du projet
  1.1 Contexte
  1.2 Vision
2. Problématique et objectifs
3. Acteurs et fonctionnalités
  3.1 Acteurs du système
  3.2. Fonctionnalités principales
  3.3 Cas d'utilisation
4. Architecture globale
  4.1 Composants de l'architecture
  4.2 Sécurité
  4.3 Principes de communication
5. Découpage en microservices
6. Conception détaillée des microservices
    6.1 User Service
    6.2 Organization Service
    6.3 Project Service
    6.4 Task Service
    6.5 Document Service
    6.6 Collaboration Service
    6.7 Audit Service
7. Diagramme de classes global
8. Interactions entre microservices
    8.1 Communications synchrones (REST / OpenFeign)
    8.2 Communications asynchrones (RabbitMQ)
9. Technologies utilisées
10. Planning et gestion des risques
    10.1 Étapes de développement
    10.2 Difficultés potentielles et solutions
11.Conclusion

1.Présentation du projet

1.1 Contexte:

Dans un environnement professionnel où plusieurs entreprises peuvent intervenir sur un même projet, la coordination des équipes, le partage des documents et le suivi des tâches représentent des enjeux importants.
Chaque entreprise utilise généralement ses propres outils de communication et de gestion. Cette dispersion peut compliquer les échanges, ralentir le suivi des projets et rendre difficile la traçabilité des actions effectuées.
C'est dans ce contexte que s'inscrit le projet **Nexus.**

1.2 Vision:

Nexus donne à chaque entreprise un espace sécurisé. Elle y invite ses partenaires, ouvre des projets communs, partage des documents avec des droits précis, échange par messagerie et suit l'avancement des tâches. Chaque action importante déclenche une notification aux personnes concernées.

2. Problématique et objectifs:

comment permettre à plusieurs entreprises de collaborer efficacement et en sécurité, sans multiplier les outils ni exposer plus de données que nécessaire ?
Problèmes constatés:

- Dispersion des échanges et des fichiers entre plusieurs outils, sans historique unique.
- Absence de cadre clair pour inviter un partenaire externe et limiter ce qu'il voit.
- Suivi des tâches et des échéances difficile entre équipes de sociétés différentes.
- Versions de documents multiples, avec le risque de travailler sur un fichier obsolète.
- Information tardive : personne n'est prévenu quand une tâche change ou qu'un document est déposé.

Objectifs fonctionnels:

Centraliser les entreprises, leurs membres et leurs partenariats dans un annuaire fiable.
Permettre la création de projets communs avec tâches, responsables et jalons.
Partager des documents avec versionnage et droits de lecture ou d'écriture par partenaire.
Offrir une messagerie intégrée, par projet et en direct.
Notifier automatiquement les acteurs concernés des événements importants.

Objectifs techniques:

Découper le système en services indépendants, chacun avec sa base de données.
Sécuriser l'accès par authentification JWT et contrôle des rôles à la passerelle.
Faire communiquer les services en synchrone (REST) pour les lectures et en asynchrone (événements) pour les notifications.
Documenter chaque API avec OpenAPI et automatiser le déploiement avec Docker.
Assurer la résilience : un service en panne ne bloque pas les autres.

3. Acteurs et fonctionnalités

3.1 Acteurs du système

Quatre acteurs interagissent avec NEXUS. Les deux premiers agissent à l'échelle de la plateforme ou de leur entreprise ; les deux derniers agissent à l'échelle d'un projet.

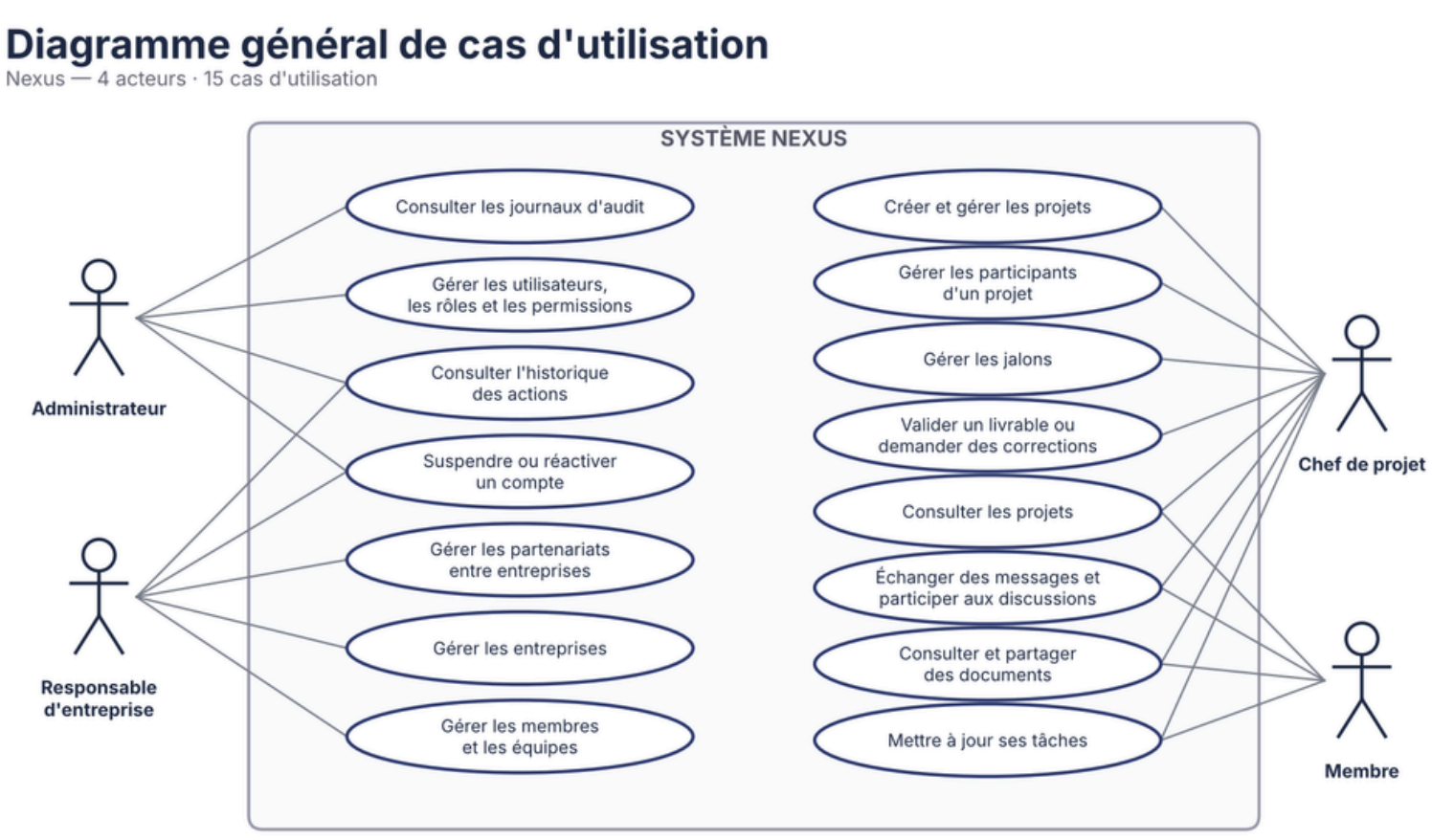
Acteur	Rôle technique	Responsabilité	Droits principaux	Services
Administrateur	ROLE_ADMIN	Gestion globale de la plateforme	Gérer utilisateurs et rôles, consulter l'audit	user, audit
Responsable d'entreprise	ROLE_ORG_MANAGER	Gestion de l'organisation, des membres et des équipes	Gérer organisation, équipes, membres et partenariats	organisation
Chef de projet	ROLE_PROJECT_MANAGER	Coordination des projets et suivi des tâches	Gérer projets, jalons, tâches et partage de documents	project, task, document
Membre	ROLE_MEMBER	Participation aux projets et réalisation des tâches	Mettre à jour ses tâches, déposer des documents, discuter	task, document, collaboration

3.2. Fonctionnalités principales

- **Gestion des utilisateurs – User Service**
  - Création et gestion des comptes.
  - Authentification des utilisateurs.
  - Gestion des rôles globaux.
  - Gestion des permissions.
  - Suspension ou activation des comptes.
  - Génération et renouvellement des jetons d'accès.
- **Gestion des organisations – Organization Service**
  - Création et modification des organisations.
  - Gestion des membres d'une organisation.
  - Création et gestion des équipes.
  - Affectation des membres aux équipes.
  - Gestion des partenariats entre organisations.
- **Gestion des projets – Project Service**
  - Création et modification des projets.
  - Définition des dates et des objectifs.
  - Gestion des organisations participantes.
  - Gestion des membres affectés aux projets.
  - Définition des rôles des participants.
  - Création et suivi des jalons.

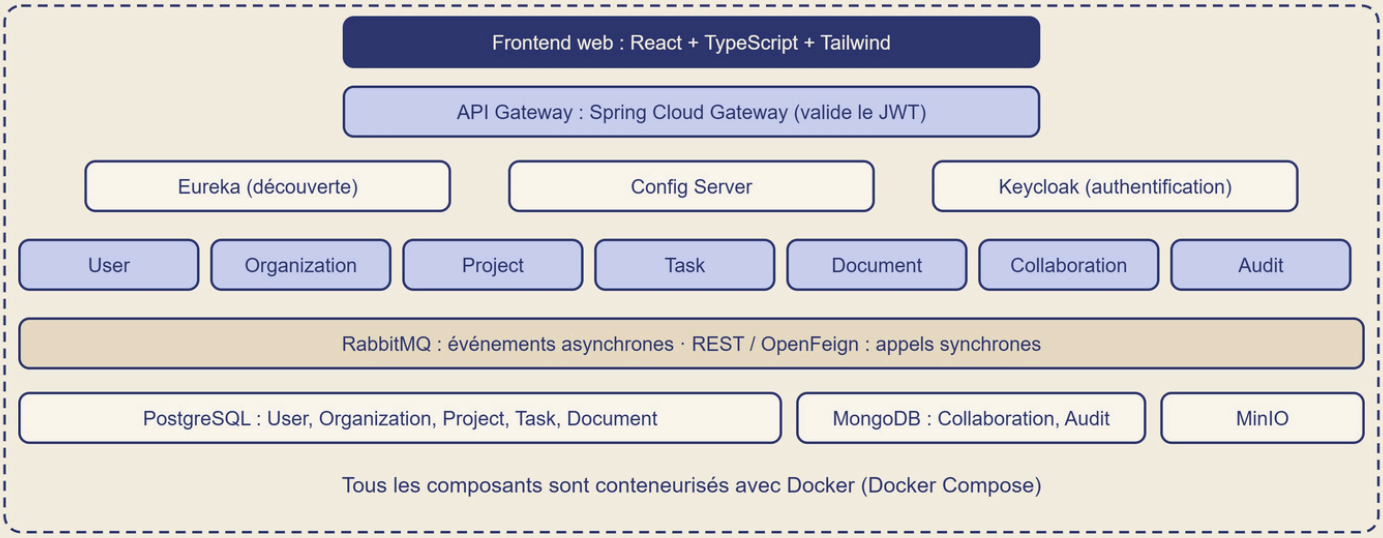
- **Gestion des tâches – Task Service**
  - Création et modification des tâches.
  - Attribution des tâches aux utilisateurs.
  - Définition des priorités.
  - Gestion des dates de début et d'échéance.
  - Suivi des statuts.
  - Gestion des dépendances entre tâches.
  - Ajout de commentaires aux tâches.
- **Gestion des documents – Document Service**
  - Dépôt de documents.
  - Consultation et téléchargement.
  - Gestion des informations des documents.
  - Gestion des autorisations d'accès.
  - Ajout de commentaires aux documents.
- **Collaboration – Collaboration Service**
  - Création d'espaces de discussion liés aux projets.
  - Envoi de messages.
  - Consultation des échanges.
  - Mention d'autres participants.
  - Association de documents aux discussions.
- **Traçabilité – Audit Service**
  - Enregistrement des actions importantes.
  - Conservation de l'historique des modifications.
  - Traçabilité des connexions.
  - Enregistrement des événements de sécurité.
  - Consultation des journaux d'audit.

3.3 Cas d'utilisation



4. Architecture globale

Les utilisateurs accèdent à la plateforme via l'interface React. Toutes les requêtes passent par l'API Gateway (Spring Cloud Gateway), qui assure le routage et la vérification des jetons JWT émis par Keycloak. Eureka assure la découverte des services et le Config Server centralise leurs configurations. Chaque microservice possède sa propre base de données (PostgreSQL ou MongoDB). Les services communiquent en REST avec OpenFeign pour les échanges synchrones, et via RabbitMQ pour les échanges asynchrones. MinIO assure le stockage des fichiers et Docker Compose permet de conteneuriser l'ensemble des composants.



4.1 Composants de l'architecture

Composant	Technologie	Rôle
Frontend web	React + TypeScript + Tailwind CSS	Interface utilisateur dynamique ; appelle uniquement l'API Gateway.
API Gateway	Spring Cloud Gateway	Point d'entrée unique, routage vers les microservices, validation du JWT.
Découverte de services	Eureka	Enregistrement des instances et résolution dynamique des adresses des services.
Configuration centralisée	Spring Cloud Config Server	Configuration externalisée des 7 microservices (profils, variables).
Authentification	Keycloak	Gestion de l'authentification et émission des JWT.

Microservices	Java 17 / Spring Boot	7 services métier indépendants : User, Organization, Project, Task, Document, Collaboration, Audit.
Communication asynchrone	RabbitMQ	Échange d'événements entre services sans attendre de réponse.
Communication synchrone	REST / OpenFeign	Appels directs lorsque une réponse immédiate est nécessaire.
Bases relationnelles	PostgreSQL	User, Organization, Project, Task, Document (une base par service).
Base documentaire	MongoDB	Collaboration, Audit (données semi-structurées).
Stockage d'objets	MinIO	Stockage des fichiers et de leurs versions (ce n'est pas une base de données).
Conteneurisation	Docker / Docker Compose	Exécution de l'ensemble des composants dans un environnement reproductible.
Stockage d'objets	MinIO	Stockage des fichiers et de leurs versions (ce n'est pas une base de données).
Conteneurisation	Docker / Docker Compose	Exécution de l'ensemble des composants dans un environnement reproductible.

4.2 Sécurité

• **Authentification :** Keycloak émet des jetons JWT contenant l'identifiant de l'utilisateur, son entreprise et son rôle.

• **Contrôle à deux niveaux :** l'API Gateway vérifie la validité du jeton, puis chaque microservice contrôle les droits avec Spring Security (annotation @PreAuthorize).

• **Rôles :** rôles globaux gérés par le User Service (Super Admin, Admin d'entreprise, Utilisateur) ; rôles de projet gérés par le Project Service.

• **Partenariats :** le droit d'accès d'une entreprise aux projets d'une autre est vérifié via l'Organization Service.

•**Documents** : autorisations vérifiées avant tout accès, URLs pré-signées MinIO à durée limitée, HTTPS, contrôle du type et de la taille des fichiers

4.3 Principes de communication

REST/OpenFeign lorsqu'une réponse immédiate est nécessaire (par exemple vérifier l'existence d'un projet avant de créer une tâche)

RabbitMQ dans les autres cas (notification d'un fait déjà accompli, par exemple l'enregistrement d'une action dans l'audit). Aucune base de données n'est partagée : les services se référencent uniquement par identifiant (UUID)

5. Découpage en microservices

Le découpage suit les contextes bornés du domaine métier : chaque microservice correspond à une capacité métier, possède ses propres données et ne recouvre pas les responsabilités d'un autre. Il ne s'agit pas d'un découpage par table ni par écran

Service	Rôle	Entités principales	Base de données
User Service (Identity)	Comptes, authentification, rôles	User, Role, UserRole, RefreshToken	PostgreSQL
Organization Service	Organisations, équipes, membres, partenariats	Organization, Team, Member, Partnership	PostgreSQL
Project Service	Projets collaboratifs, participants, jalons	Project, ProjectParticipant, ProjectMember, Milestone	PostgreSQL
Task Service	Tâches, affectations, dépendances, commentaires	Task, TaskAssignment, TaskDependency, TaskComment	PostgreSQL
Document Service	Documents, versions, droits	Document, DocumentVersion, DocumentPermission	MongoDB (métadonnées) et stockage de fichiers
Collaboration Service	Discussions, messages, mentions	Discussion, Message, Mention	MongoDB
Audit Service	Traçabilité, alertes de sécurité, exports	AuditEvent, AuditLog, AuditSecurity, AuditExport	MongoDB

6. Conception détaillée des microservices

Pour chaque microservice, cette section présente ses responsabilités, son modèle de données, le choix de sa base, son contrat d'API REST, ses événements et ses interactions. Les contrats d'API sont une proposition de conception

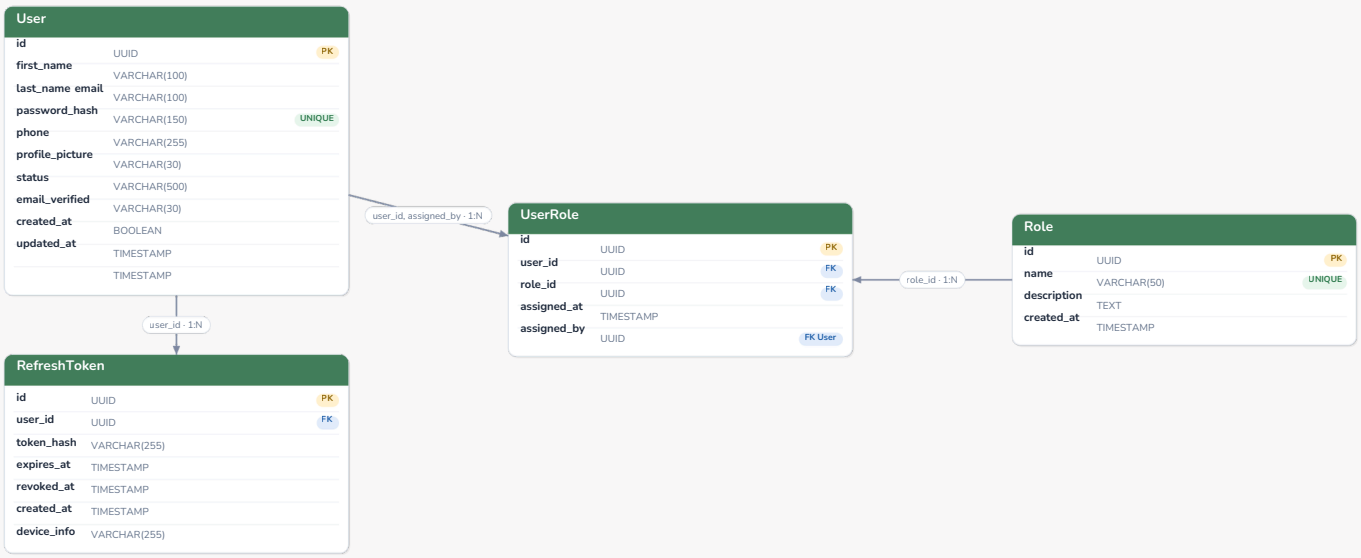
6.1 User Service (microservice commun)

Rôle. Il gère l'identité de tous les utilisateurs. Il est le service commun dont dépendent la passerelle et tous les autres services.
Responsable : toute l'équipe (présentation par un membre désigné). Base : PostgreSQL.

Responsabilités

- Inscription et gestion des profils.
- Authentification : vérification des identifiants, émission et rafraîchissement des jetons JWT.
- Attribution et retrait des rôles.

Interactions. Il est appelé par la passerelle pour la connexion. Il publie utilisateur.créé et utilisateur.connecté.



Entité	Description
User	Compte utilisateur : identité, e-mail unique, mot de passe haché, téléphone, photo, statut, e-mail vérifié.
Role	Rôle global (Super Admin, Admin d'entreprise, Utilisateur) ; nom unique.
UserRole	Association utilisateur–rôle, avec la date et l'auteur de l'attribution (assigned_by).
RefreshToken	Jeton de rafraîchissement (haché) avec expiration, révocation et information sur l'appareil.

Justification de la base : les comptes, rôles et associations sont fortement relationnels, avec des contraintes d'unicité (e-mail, nom de rôle) et des clés étrangères : PostgreSQL garantit l'intégrité et les transactions.

Contrat d'API REST

Méthode	Endpoint	Description
POST	/api/users	Créer un utilisateur (inscription)
POST	/api/auth/login	Se connecter ; retourne le jeton et le refresh token
POST	/api/auth/refresh	Renouveler le jeton
POST	/api/auth/logout	Révoquer le refresh token
GET	/api/users/{id}	Consulter un utilisateur
GET	/api/users	Lister les utilisateurs (Admin)
PUT	/api/users/{id}	Modifier le profil
PATCH	/api/users/{id}/status	Suspendre ou réactiver un compte
POST	/api/users/{id}/roles	Attribuer un rôle global
DELETE	/api/users/{id}/roles/{roleId}	Retirer un rôle global

Interactions : appelé en synchrone par l'Organization Service (vérifier l'existence d'un utilisateur) ; publie ses événements vers l'Audit Service via RabbitMQ (connexions, modifications de compte). Si l'Audit Service est indisponible, les messages restent dans une file durable et le User Service n'est pas impacté

6.2 Organization Service

Responsable : Latrech Arij. Base : PostgreSQL

Responsabilités

Gestion des entreprises, de leurs équipes, de leurs membres et des partenariats entre entreprises. Il est la référence pour vérifier qu'un partenariat est actif avant d'autoriser la collaboration



Entité	Description
Organization	Entreprise : nom, description, coordonnées, logo, statut.
Team	Équipe appartenant à une entreprise ; créée par un utilisateur (référence au User Service).
Member	Membre d'une entreprise (et éventuellement d'une équipe) ; référence l'utilisateur par user_id ; poste, statut, date d'arrivée.
Partnership	Partenariat entre deux entreprises (organization_id, partner_organization_id) : invitation, statut, date d'acceptation.

Justification de la base : les relations entreprise–équipes–membres–partenariats sont hiérarchiques et nécessitent des contraintes d'intégrité : PostgreSQL est adapté.

Contrat d'API REST

Méthode	Endpoint	Description
POST	/api/organizations	Créer une entreprise
GET	/api/organizations/{id}	Consulter une entreprise
PUT	/api/organizations/{id}	Modifier une entreprise
POST	/api/organizations/{id}/teams	Ajouter une équipe
GET	/api/organizations/{id}/teams	Lister les équipes d'une
POST	/api/organizations/{id}/members	Ajouter un membre (vérifie
DELETE	/api/organizations/{id}/members/{	Retirer un membre
POST	/api/partnerships	Inviter une entreprise partenaire
PATCH	/api/partnerships/{id}/status	Accepter ou refuser un

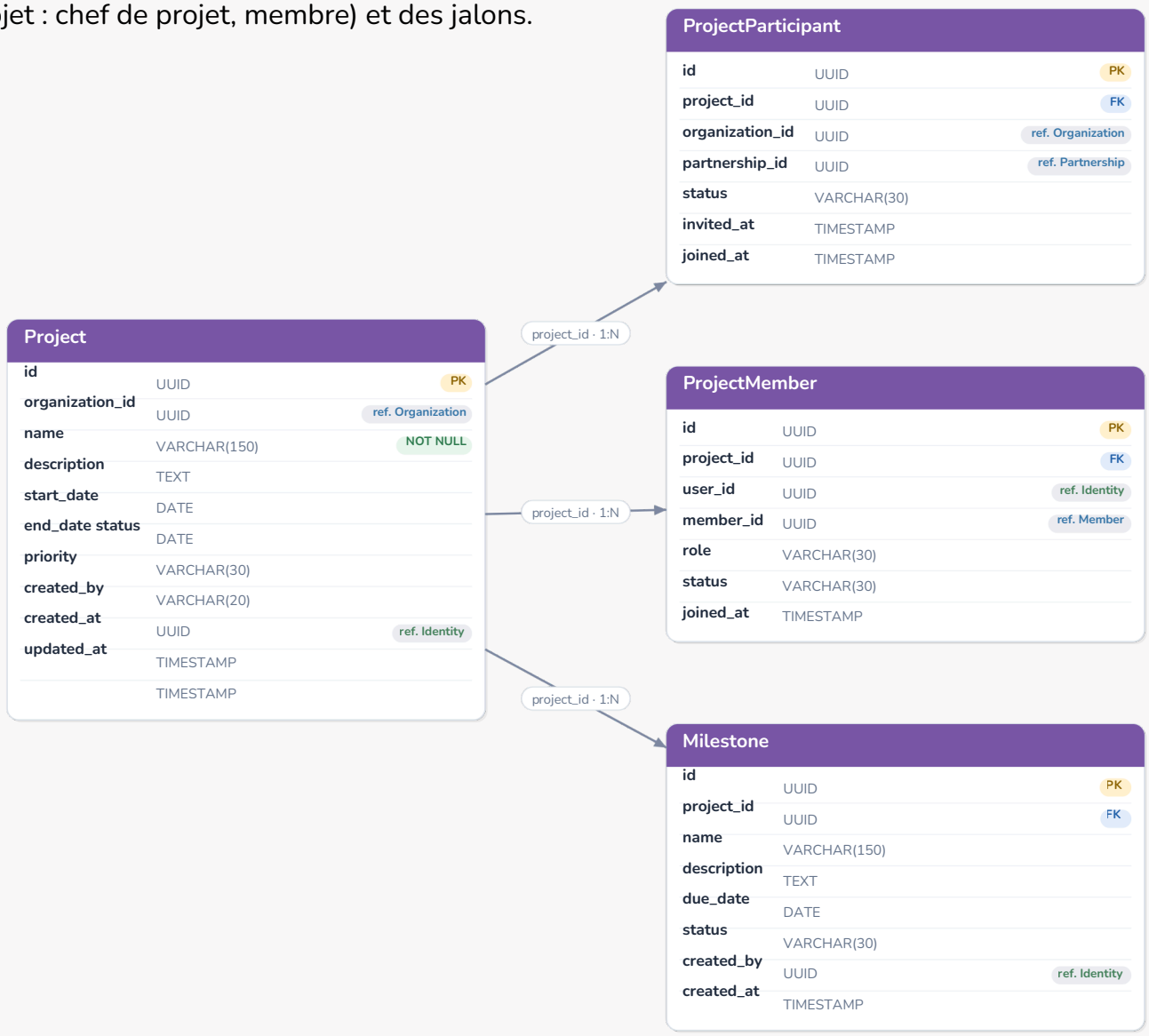
Interactions : appelle le User Service en synchrone (OpenFeign) pour vérifier l'existence d'un utilisateur avant de l'ajouter comme membre ; est appelé par le Project Service pour vérifier une entreprise partenaire ; publie ses événements vers l'Audit Service (asynchrone).

6.3 Project Service

Responsable : Dawahi Med Aziz. Base : PostgreSQL.

Responsabilités

Création et gestion des projets, des entreprises participantes, des membres de projet (avec leurs rôles de projet : chef de projet, membre) et des jalons.



Entité	Description
Project	Projet porté par une entreprise : nom, description, dates, statut, priorité.
ProjectParticipant	Entreprise participant au projet, via un partenariat (organization_id, partnership_id) ; statut d'invitation.
ProjectMember	Utilisateur membre du projet avec son rôle de projet et son statut.
Milestone	Jalon du projet : nom, description, échéance, statut.

Justification de la base : projets, participants, membres et jalons sont liés par des clés étrangères et nécessitent des requêtes de filtrage et de tri : PostgreSQL est adapté.

Contrat d'API REST

Méthode	Endpoint	Description
POST	/api/projects	Créer un projet
GET	/api/projects	Lister les projets accessibles
GET	/api/projects/{id}	Consulter un projet
PUT	/api/projects/{id}	Modifier un projet
PATCH	/api/projects/{id}/status	Changer le statut du projet
POST	/api/projects/{id}/participants	Inviter une entreprise partenaire
POST	/api/projects/{id}/members	Ajouter un membre au projet
DELETE	/api/projects/{id}/members/{memberId}	Retirer un membre
POST	/api/projects/{id}/milestones	Créer un jalon
PUT	/api/milestones/{id}	Modifier un jalon

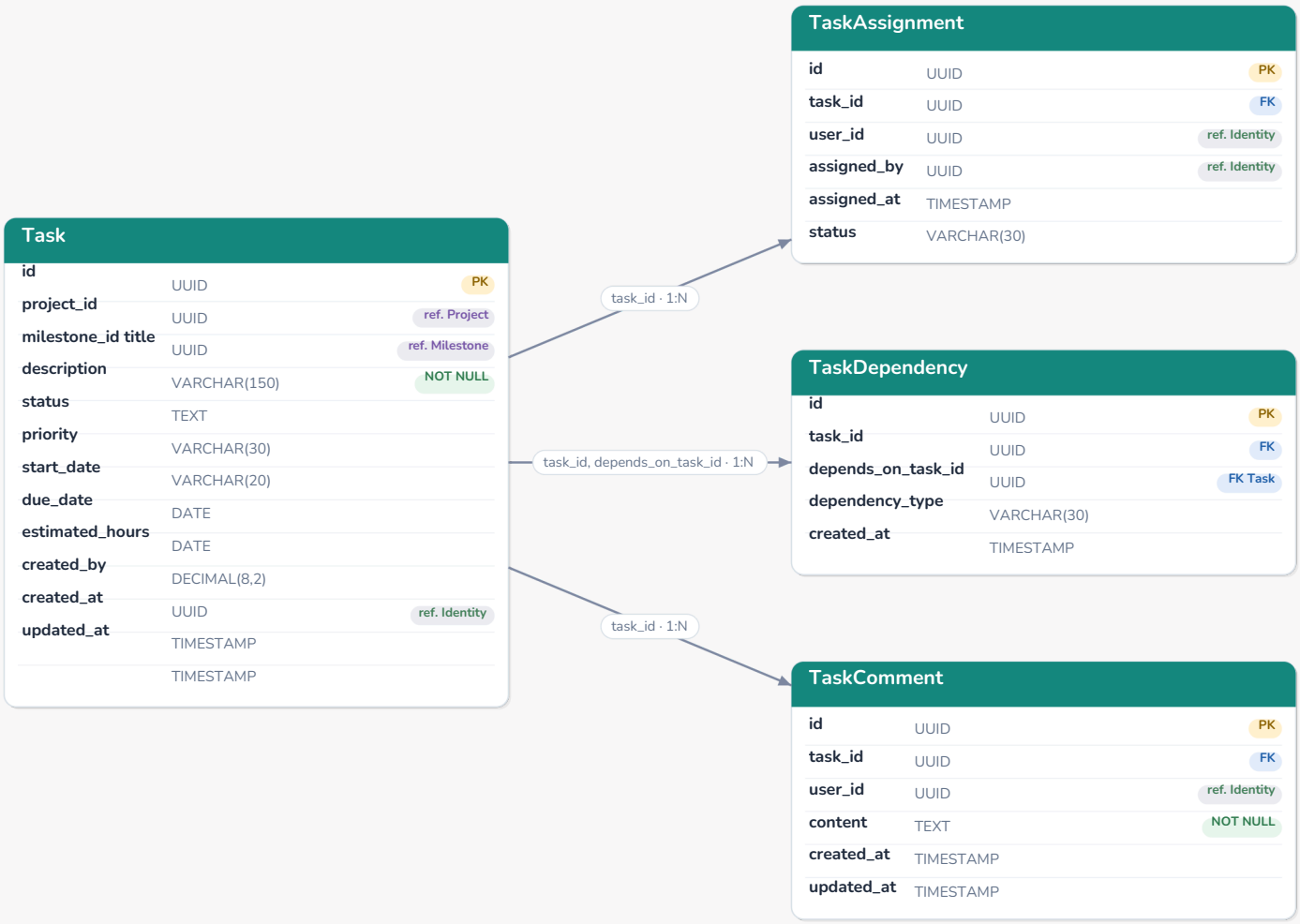
Interactions : appelle l'Organization Service en synchrone pour vérifier une entreprise partenaire ; est appelé par les services Task, Document et Collaboration pour vérifier l'existence d'un projet ; publie ses événements vers l'Audit Service

6.4 Task Service

Responsable : Mazitti Chaima. Base : PostgreSQL.

Responsabilités

Création, attribution et suivi des tâches d'un projet : priorités, échéances, estimation de charge, dépendances entre tâches et commentaires.



Entité	Description
Task	Tâche d'un projet (et éventuellement d'un jalon) : titre, description, statut, priorité, dates, heures estimées.
TaskAssignment	Attribution d'une tâche à un utilisateur : auteur et date d'attribution, statut.
TaskDependency	Dépendance entre deux tâches (depends_on_task_id) et son type.
TaskComment	Commentaire rattaché à une tâche.

Justification de la base : les dépendances entre tâches et les attributions forment un graphe relationnel avec des contraintes d'intégrité : PostgreSQL est adapté.

Méthode	Endpoint	Description
POST	/api/tasks	Créer une tâche (vérifie le projet)
GET	/api/tasks?projectId={id}	Lister les tâches d'un projet
GET	/api/tasks/{id}	Consulter une tâche
PUT	/api/tasks/{id}	Modifier une tâche
PATCH	/api/tasks/{id}/status	Mettre à jour le statut (ex. terminée)
POST	/api/tasks/{id}/assignments	Attribuer la tâche à un utilisateur
POST	/api/tasks/{id}/dependencies	Ajouter une dépendance
POST	/api/tasks/{id}/comments	Commenter une tâche
DELETE	/api/tasks/{id}	Supprimer une tâche

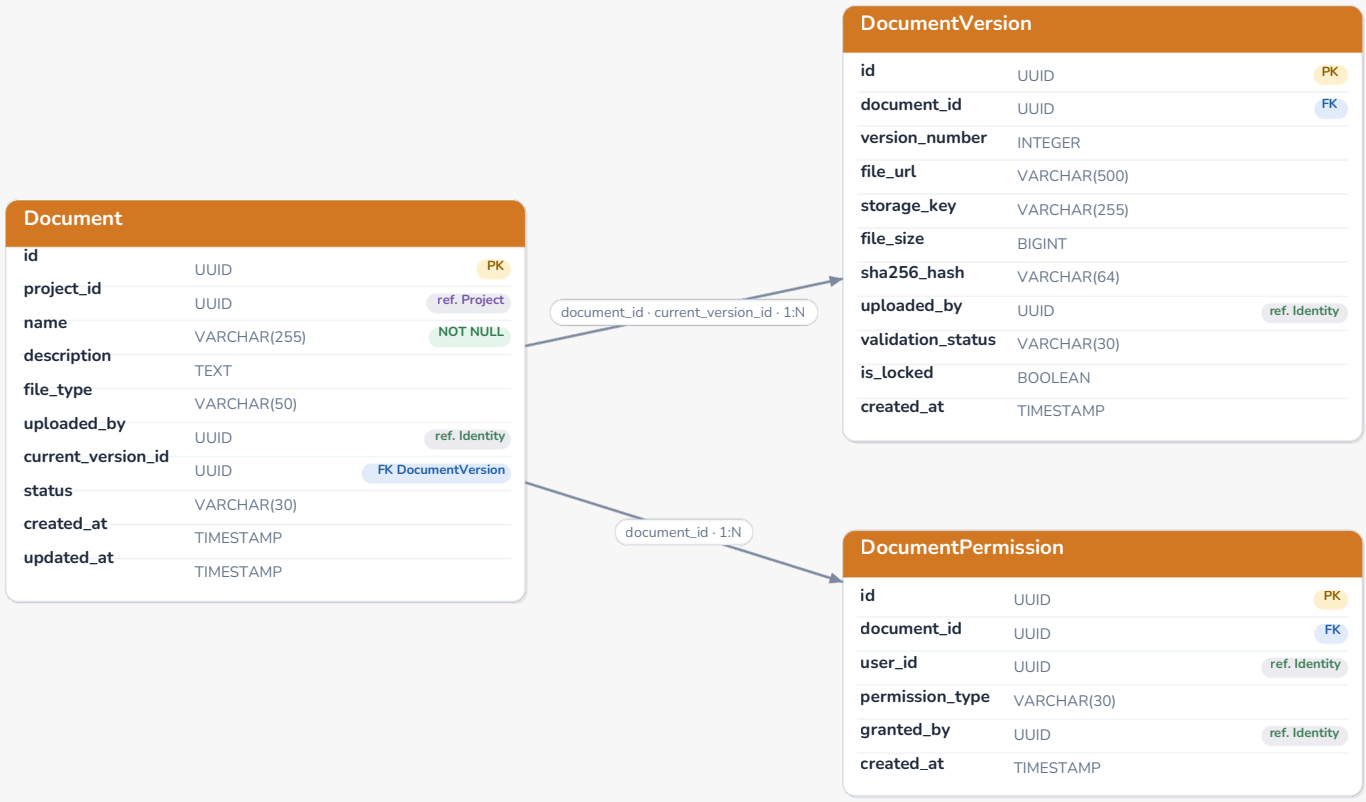
Interactions : appelle le Project Service en synchrone pour vérifier l'existence du projet associé à une tâche ; est appelé par le Collaboration Service ; publie ses événements vers l'Audit Service

6.5 Document Service

Responsable : Mihoubi Akrem. Base : PostgreSQL (métadonnées) et MinIO (fichiers).

Responsabilités

Dépôt, partage et versionnement des documents, gestion des autorisations d'accès et de l'état de validation de chaque version (validation d'un livrable ou demande de corrections). Les fichiers sont stockés dans MinIO ; seules leurs métadonnées sont dans PostgreSQL.



Entité	Description
Document	Document d'un projet : nom, description, type de fichier, auteur du dépôt, référence vers la version
DocumentVersion	Version d'un document : numéro, clé de stockage MinIO, taille, empreinte SHA-256, statut de
DocumentPermission	Autorisation accordée à un utilisateur sur un document (type de permission, accordée par).

Justification de la base : les métadonnées, les versions et les permissions sont relationnelles (PostgreSQL) ; le contenu binaire est externalisé dans MinIO, stockage d'objets adapté aux fichiers volumineux et versionnés. L'empreinte SHA-256 garantit l'intégrité des versions.

Contrat d’API REST

Méthode	Endpoint	Description
POST	/api/documents	Déposer un document (multipart)
GET	/api/documents?projectId={id}	Lister les documents d’un projet
GET	/api/documents/{id}	Consulter les métadonnées
POST	/api/documents/{id}/versions	Ajouter une nouvelle version
GET	/api/documents/{id}/versions	Lister les versions
GET	/api/documents/{id}/versions/{n}/download-url	Obtenir une URL pré-signée MinIO à durée limitée
PATCH	/api/documents/{id}/versions/{n}/validation	Valider un livrable ou demander des corrections
POST	/api/documents/{id}/permissions	Accorder une permission
DELETE	/api/documents/{id}/permissions/{permissionId}	Révoquer une permission

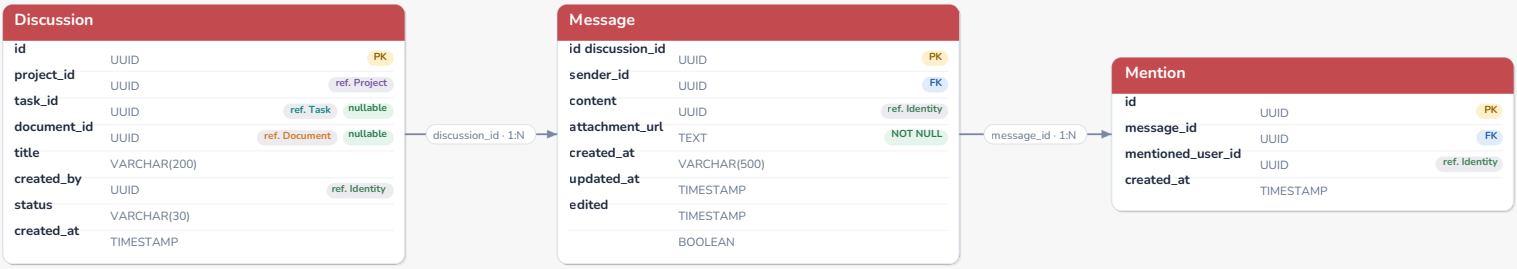
Interactions : appelle le Project Service en synchrone pour vérifier le projet auquel appartient le document ; est appelé par le Collaboration Service ; publie ses événements vers l’Audit Service.

6.6 Collaboration Service

Responsable : Ben Boubaker Sarra. Base : MongoDB.

Responsabilités

Gestion des discussions rattachées à un projet, à une tâche ou à un document, des messages échangés et des mentions de collègues.



Entité	Description
Discussion	Fil de discussion rattaché à un projet, et éventuellement à une tâche ou à un document (références facultatives) ; titre, statut.
Message	Message d'une discussion : expéditeur, contenu, pièce jointe éventuelle, indicateur de modification.
Mention	Mention d'un utilisateur dans un message.

Justification de la base : messages, discussions et mentions sont des données semi-structurées, nombreuses et écrites fréquemment ; le modèle documentaire de MongoDB (une discussion et ses messages) évite des jointures coûteuses et passe bien à l'échelle. Les types du schéma ci-dessus (UUID, TIMESTAMP, etc.) sont le modèle conceptuel, à traduire en documents MongoDB.

Contrat d'API REST

Méthode	Endpoint	Description
POST	/api/discussions	Créer une discussion (vérifie projet, tâche ou document)
GET	/api/discussions?projectId={id}	Lister les discussions (filtres : tâche, document)
PATCH	/api/discussions/{id}/status	Clôturer ou rouvrir une discussion

POST	/api/messages	Envoyer un message (avec mentions éventuelles)
GET	/api/discussions/{id}/messages	Lister les messages d'une discussion
PUT	/api/messages/{id}	Modifier un message
DELETE	/api/messages/{id}	Supprimer un message

Interactions : appelle en synchrone le Project Service, le Task Service et le Document Service pour vérifier l'existence de l'élément auquel une discussion est rattachée ; publie ses événements vers l'Audit Service

6.7 Audit Service

Responsable : Ben Hamouda Yoser. Base : MongoDB.

Responsabilités

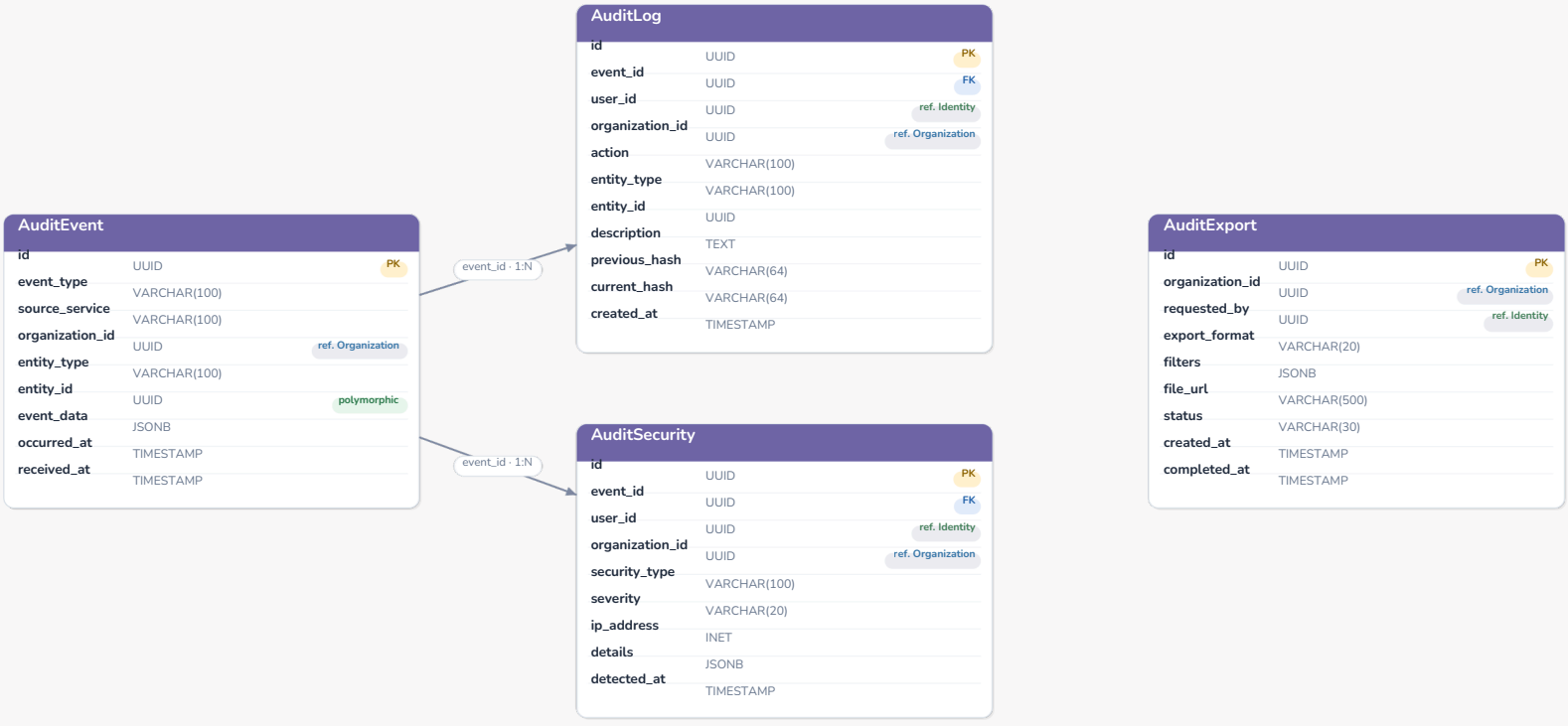
Enregistrement des événements publiés par les autres services (connexion, changement de droits, dépôt ou suppression de document, clôture de projet).

Écriture seule : une entrée du journal n'est jamais modifiée.

Détection et enregistrement des alertes de sécurité (par exemple plusieurs échecs de connexion).

Consultation, filtrage et export du journal, réservés à l'administrateur de la plateforme.

Interactions. Il n'est appelé par aucun autre service : il écoute uniquement les événements. Une panne de l'audit ne bloque donc jamais les autres services.



Entité	Description
AuditEvent	Événement brut reçu : type, service source, entreprise, entité concernée (référence polymorphe), données de l'événement, dates d'occurrence et de réception.
AuditLog	Entrée du journal : utilisateur, action, entité, description, avec empreintes chaînées (previous_hash, current_hash) pour détecter toute modification.
AuditSecurity	Événement de sécurité : type, gravité, adresse IP, détails.
AuditExport	Demande d'export de l'historique : format, filtres, statut, fichier produit.

Justification de la base : les données d'audit sont très volumineuses, écrites en ajout seul et de structure variable (le contenu de l'événement dépend du service source) : MongoDB convient à ce profil. Le chaînage d'empreintes (previous_hash / current_hash) rend toute altération détectable. Les types du schéma (JSONB, INET, etc.) sont conceptuels et à traduire en documents MongoDB.

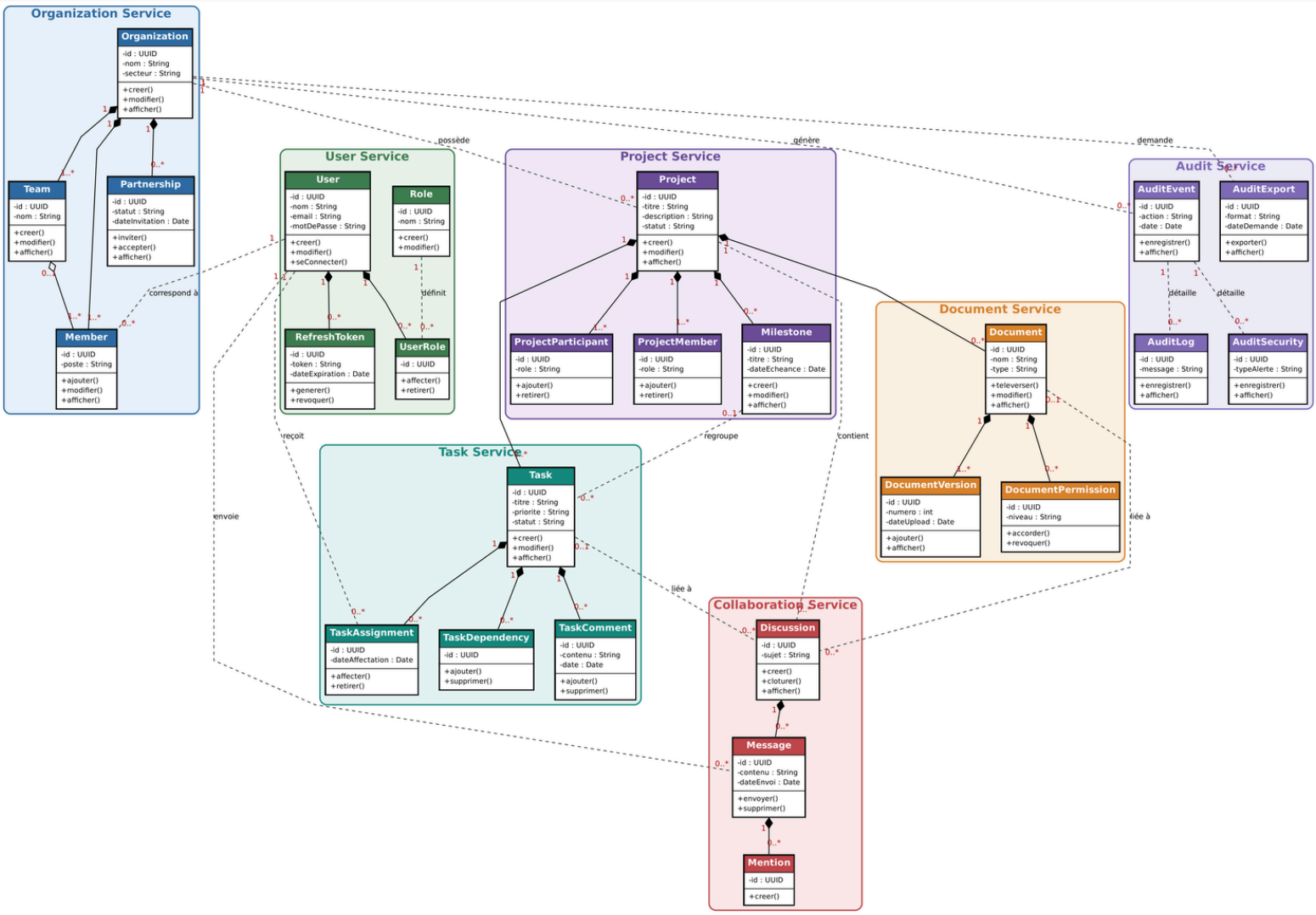
Contrat d'API REST

Méthode	Endpoint	Description
GET	/api/audit/logs	Consulter l'historique (filtres : entreprise, utilisateur, période, action)
GET	/api/audit/logs/{id}	Consulter une entrée du journal
GET	/api/audit/security	Consulter les événements de sécurité
GET	/api/audit/logs/integrity	Vérifier l'intégrité de la chaîne d'empreintes
POST	/api/audit/exports	Demander un export de l'historique
GET	/api/audit/exports/{id}	Consulter l'état d'un export

L'Audit Service n'expose volontairement aucune route de modification ou de suppression : le journal est alimenté uniquement par les événements RabbitMQ.

Interactions : exclusivement asynchrones en entrée (RabbitMQ). En cas de panne de l'Audit Service : les messages restent dans des files durables ; en cas d'échec de traitement répété ils sont placés dans une dead-letter queue pour retraitement. Les services métier ne sont pas bloqués. Les consommateurs sont idempotents (un événement rejoué n'est pas enregistré deux fois).

7. Diagramme de classes global



8. Interactions entre microservices

8.1 Communications synchrones (REST / OpenFeign)

Utilisées lorsque le service appelant a besoin d'une réponse immédiate pour poursuivre (validation d'une référence).

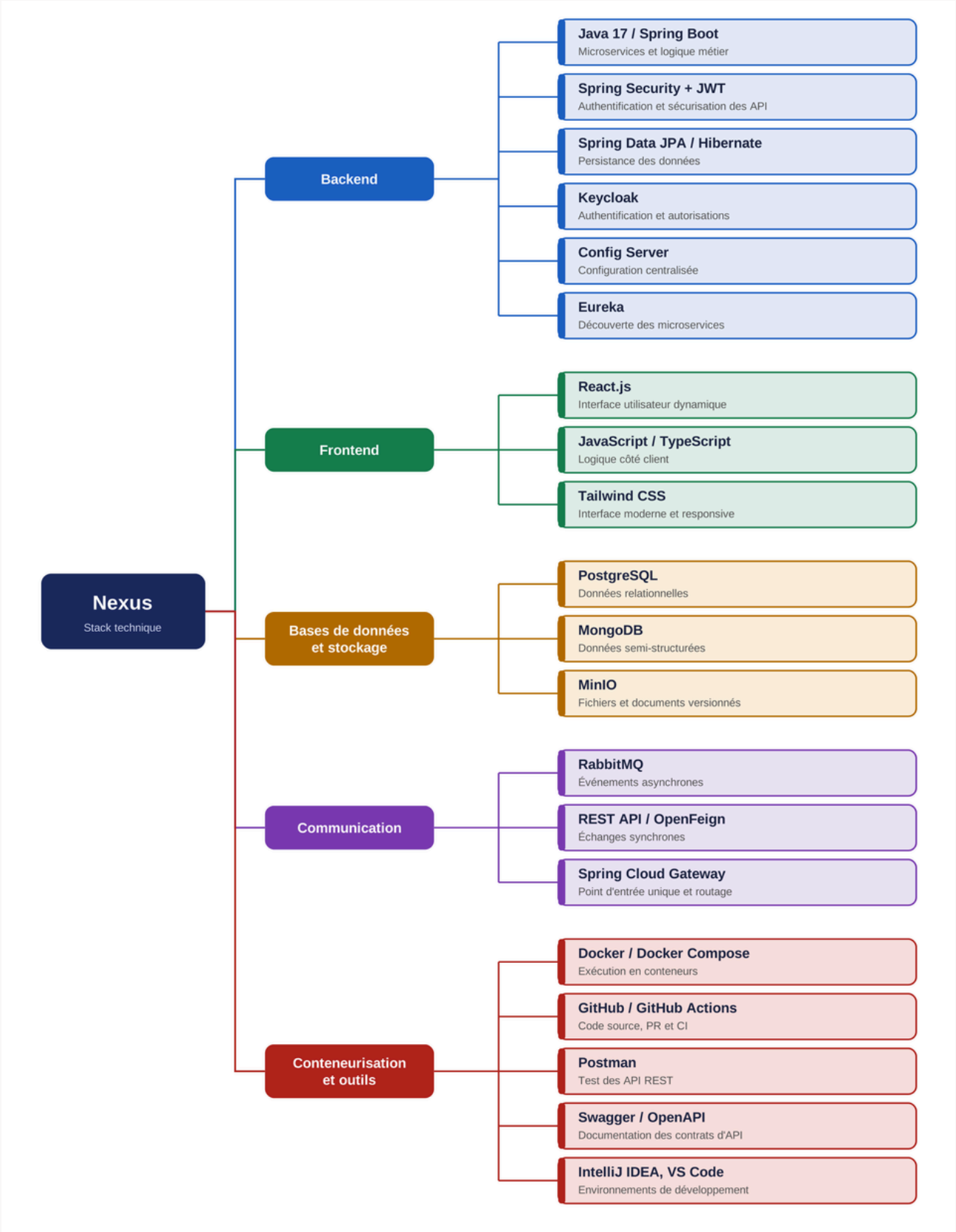
Source	Destination	Exemple
Organization	User	Vérifier l'existence d'un utilisateur avant de l'ajouter
Project	Organization	Vérifier l'existence d'une entreprise partenaire.
Task	Project	Vérifier l'existence du projet associé à une tâche.
Document	Project	Vérifier l'existence du projet auquel appartient le document.
Collaboration	Project	Vérifier l'existence du projet associé à une discussion.
Collaboration	Task	Vérifier l'existence d'une tâche associée à une discussion.
Collaboration	Document	Vérifier l'existence d'un document associé à une

8.2 Communications asynchrones (RabbitMQ)

Utilisées pour notifier un fait déjà accompli, sans attendre de réponse. Tous les services métier publient vers l'Audit Service.

Source	Destination	Événements	Exemple
User	Audit	AccountCreated, AccountSuspended	Enregistrer une connexion ou une
Organization	Audit	OrganizationCreated, OrganizationUpdated	Enregistrer la création ou la modification d'une
Project	Audit	ProjectCreated, ProjectUpdated	Enregistrer la création ou la modification d'un
Task	Audit	TaskCreated, TaskCompleted	Enregistrer la création ou la finalisation d'une
Document	Audit	DocumentUploaded, DocumentVersionAdde	Enregistrer le dépôt ou la modification d'un
Collaboration	Audit	CommentAdded, MentionCreated	Enregistrer l'envoi d'un message ou la création

9. Technologies utilisées



10. Planning et gestion des risques

10.1 Étapes de développement

Phase	Contenu
1. Analyse des besoins	Identification des besoins fonctionnels, des acteurs et des exigences du système.
2. Conception et architecture	Modélisation UML, définition de l'architecture microservices, conception des bases de données.
3. Développement Backend	Implémentation des 7 microservices, des API REST et des mécanismes de sécurité.
4. Développement Frontend	Conception de l'interface utilisateur et intégration avec les services Backend.
5. Intégration des services	Configuration de l'API Gateway et mise en place de la communication entre les microservices.
6. Tests et validation	Tests unitaires, d'intégration et fonctionnels pour vérifier la fiabilité du système.
7. Déploiement	Conteneurisation avec Docker, configuration de l'environnement et mise en service.

10.2 Difficultés potentielles et solutions

Défi	Solution
Coordination des 7 microservices	Contrats d'API documentés avec Swagger/OpenAPI, conventions communes (noms d'événements, format des erreurs), workflow GitHub (branches + pull requests), réunions de synchronisation régulières.
Cohérence des données entre bases indépendantes	Une base par service, références par identifiant (UUID) au lieu de jointures, événements RabbitMQ pour synchroniser, cohérence éventuelle, consommateurs idempotents.
Gestion des droits d'accès entre entreprises	Rôles globaux (User Service) + rôles de projet (Project Service), JWT contenant userId, entreprise et rôle, vérification à l'API Gateway puis dans chaque service (@PreAuthorize), contrôle du partenariat via l'Organization Service.
Sécurisation du partage des documents	Vérification des autorisations avant tout accès, URLs pré-signées MinIO à durée limitée, HTTPS, contrôle du type et de la taille des fichiers, versionnage des documents.
Échanges synchrones et asynchrones	Règle claire : REST quand une réponse immédiate est nécessaire, RabbitMQ sinon. Timeouts, retry et circuit breaker (Resilience4j), dead-letter queues pour les messages en échec.
Centralisation et traçabilité des actions	Audit Service qui consomme tous les événements, format d'événement standard (qui, quoi, quand, ressource), journal en ajout seul (non modifiable), identifiant de corrélation entre services.
Intégration et déploiement de l'architecture distribuée	Conteneurisation Docker de chaque service, Docker Compose pour l'environnement complet, configuration par variables d'environnement, tests d'intégration, pipeline CI avec GitHub Actions.

11. Conclusion

Le projet NEXUS vise à faciliter la collaboration entre plusieurs entreprises grâce à une plateforme centralisée permettant de gérer les projets, les tâches, les documents et les échanges.
L'architecture microservices permet de séparer les fonctionnalités, de faciliter la communication entre les services et d'assurer une meilleure évolutivité et maintenabilité de la plateforme. Chaque service dispose de sa propre base, de responsabilités claires et d'un contrat d'API défini ; les interactions synchrones et asynchrones sont identifiées et justifiées.
Cette conception constitue une base solide pour le développement d'une solution collaborative, sécurisée et évolutive. La prochaine étape consiste à implémenter la logique métier de chaque microservice, puis à les intégrer derrière l'API Gateway.